

Guide de gestion de session React ↔ PHP

Vue d'ensemble

Cette documentation explique comment la gestion de session a été implémentée pour permettre une communication fluide entre votre frontend React et vos APIs PHP.

Architecture

React Frontend (port 3007)

 (CORS + Session Cookie)

PHP Backend APIs

 (Session centralisée)

Bootstrap.php (Configuration)

 (Base de données)

MySQL

Composants principaux

1. Bootstrap.php - Session centralisée

Le fichier `bootstrap.php` configure automatiquement :

- **Session cross-domain** avec détection automatique de l'origine
- **Configuration CORS** dynamique selon l'environnement
- **Cookies sécurisés** avec `SameSite=None` pour cross-domain
- **Transmission d'ID de session** via URL (`?PHPSESSID=xxx`)

2. Auth-API.php - Authentification

Gère l'authentification avec :

- **Routes publiques** : login, reset password, debug session
- **Routes protégées** : logout, check session, administration
- **Vérification automatique** de session pour toutes les routes protégées

3. Tarif-API.php - API métier protégée

Maintenant protégée avec :

- **Vérification d'authentification** obligatoire pour toutes les routes
- **Contrôle des droits** (admin/gestionnaire) pour les modifications
- **Logging détaillé** des accès et erreurs

Flux d'authentification

Connexion (Login)

php

POST /api/auth-api.php?login

```
{  
  "username": "user",  
  "password": "pass"  
}
```

Réponse → {

```
  "success": true,  
  "user": {...}  
}
```

+ Cookie PHPSESSID défini

Vérification de session

php

GET /api/auth-api.php?check_session

Réponse → {

```
  "success": true,  
  "user": {...}  
}
```

Accès aux tarifs (protégé)

php

GET /api/tarif-api.php?services

Headers: Cookie: PHPSESSID=abc123...

Réponse → {

```
  "success": true,  
  "data": [...]  
}
```

Gestion des cookies

Côté React

javascript

```
// Configuration axios pour inclure les cookies
axios.defaults.withCredentials = true;

// Les cookies PHPSESSID sont automatiquement inclus
const response = await axios.get('/api/tarif-api.php?services');
```

Côté PHP

php

```
// Configuration automatique dans bootstrap.php
session_set_cookie_params([
    'lifetime' => 3600,
    'path' => '/',
    'domain' => '.dev-facturation', // Détecté automatiquement
    'secure' => true,
    'httponly' => false, // Accessible à React
    'samesite' => 'None' // Nécessaire pour cross-domain
]);
```

Sécurité implémentée

1. Vérification d'authentification

php

```
// Dans tarif-api.php
if (!isset($_SESSION['user_id'])) {
    http_response_code(401);
    echo json_encode([
        'success' => false,
        'message' => 'Authentification requise'
    ]);
    exit;
}
```

2. Contrôle des droits

php

```
// Pour les modifications (POST/PUT/DELETE)
if ($userRole !== 'admin' && $userRole !== 'gestionnaire') {
    throw new Exception('Droits administrateur requis', 403);
}
```

3. Validation d'ID de session

php

```
// Protection contre injection
if (preg_match('/^[a-zA-Z0-9-]{20,40}$/i', $_GET['PHPSESSID'])) {
    session_id($_GET['PHPSESSID']);
} else {
    error_log("⚠️ SÉCURITÉ - ID session invalide");
}
```

Debug et diagnostic

Route de debug

php

GET /api/tarif-api.php?debug_session

// En mode développement uniquement

```
Réponse → {
  "success": true,
  "debug_info": {
    "session_id": "abc123...",
    "user_authenticated": true,
    "user_id": 1,
    "session_data": {...}
  }
}
```

Logs détaillés

En mode développement, tous les accès sont loggés :

tarif-api -  Accès autorisé pour utilisateur: 1 (role: admin)
tarif-api - Récupération services pour user: 1

Configuration React

Axios configuration

javascript

```
// Configurer axios pour les cookies cross-domain
axios.defaults.withCredentials = true;
axios.defaults.baseURL = 'https://dev-facturation-api';

// Intercepteur pour gérer les erreurs d'authentification
axios.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // Rediriger vers login
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```

Points clés d'implémentation

1. Session unifiée

- Toutes les APIs utilisent `bootstrap.php`
- Session configurée automatiquement
- ID de session transmis via URL si nécessaire

2. CORS dynamique

- Détection automatique de l'origine
- Configuration adaptée (dev vs prod)
- Headers sécurisés

3. Protection par défaut

- Toutes les routes tarif nécessitent une authentification
- Droits vérifiés pour les modifications
- Erreurs détaillées en développement

4. Compatibilité React

- `httponly: false` pour accès JavaScript
- `withCredentials: true` dans axios
- Gestion automatique des cookies

Avantages de cette approche

1. **Sécurisé** : Authentification obligatoire, contrôle des droits
2. **Unifié** : Même système pour toutes les APIs
3. **Compatible** : Fonctionne avec React et JavaScript
4. **Debuggable** : Logs détaillés et route de diagnostic
5. **Évolutif** : Facile d'ajouter de nouvelles APIs protégées

Pour ajouter une nouvelle API protégée

1. Inclure `bootstrap.php`
2. Appeler `init_api_response()`
3. Vérifier `$_SESSION['user_id']`
4. Contrôler les droits selon les besoins
5. Ajouter du logging en mode dev

Cette architecture garantit une sécurité cohérente tout en maintenant la flexibilité pour votre application React.